MMBasic Performance Guide

How to make MMBasic programs run faster: measured on the PicoMite, MMBasic V7.0.00b7

Introduction

This guide answers the questions people ask about writing fast MMBasic. Should statements go one per line or several? Is FOR faster than DO? `NEXT` or `NEXT x`? Numbers, constants or variables? Is `CALL "sub"` faster than calling the SUB? Do long names slow a program down? It then covers what else makes a difference.

Every answer is measured. Each construct was timed on an RP2040 and an RP2350 running V7.0.00b7, in each of the three ways V7 can run a program:

- **interpreted, with symbols**: the default, on every board;
- **interpreted, without symbols**: with `OPTION SYMBOLS OFF` at the top of the program;
- **compiled**: with `OPTION COMPILE ON` (RP2350 only).

The programs that took the measurements are in the MMBasic source tree, in `Bas/bench/perfguide/` (Appendix B). Run them on your own board to see its numbers.

The manual *OPTION PROFILING and Built-in Optimisations* (option-profiling-cache.pdf) describes the profiler and the optimisations built into the interpreter. This guide is about the program you write.

1. Where the time goes

MMBasic keeps your program in a compact form. Each keyword is stored as a one- or two-byte token. Since V7, each name (variable, constant, SUB, FUNCTION or label) is stored as a two- or three-byte *symbol*. Everything else stays as you typed it: numbers as their digits, operators, brackets, spaces and comments.

Each time a statement runs, the interpreter reads it again from the start:

- it finds the command;
- it reads every argument through the expression evaluator;
- it finds every variable;
- it converts every number from its digits;
- when the statement is done, it checks for Ctrl-C and for interrupts.

That reading is almost all of the time a statement takes. The arithmetic is a small part of it: `a% = b% * c%` and `a = b * c` take the same time, and on an RP2350 a `SIN` costs little more than a `+`.

So the rules for fast MMBasic are few:

1. **Fewer statements.** A statement costs several microseconds before it does anything useful. One statement that does more work is cheaper than two that do half each.
2. **Less to read in each statement.** Every name, number, operator and bracket costs time. Spaces and comments at the end of a statement cost a little too.
3. **Let the firmware do the loop.** A command that works on a whole array does in C what a BASIC loop does one statement at a time, and is typically 20 to 100 times faster. Such commands are `MATH`, `ARRAY`, `SORT`, `MEMORY` and the array forms of `PIXEL`, `LINE` and `BOX`.
4. **On an RP2350, compile.** `OPTION COMPILE ON` removes most of the reading. Simple numeric statements run 8 to 14 times faster.

How to read the tables

Times are in microseconds (µs). Most tests time a statement inside a `FOR i = 1 TO 20000 ... NEXT` loop. The tables give the time of **the statement alone**: the time per pass, less the cost of the empty loop. The empty loop takes 2.16 µs on the RP2040, 2.28 µs on the RP2350 and 1.12 µs compiled. Tests of the loops themselves give the whole time per pass.

Column	Board and mode
RP2040	PicoMiteVGA (RP2040) at 378 MHz, symbols on (the default)
RP2040 off	the same, with `OPTION SYMBOLS OFF`
RP2350	PicoMiteHDMIWEB (RP2350B, a PicoComputer 3) at 252 MHz, symbols on, `OPTION COMPILE OFF`
RP2350 off	the same, with `OPTION SYMBOLS OFF`
Compiled	the same RP2350, with `OPTION COMPILE ON`

The two boards ran at different clocks, so compare figures within a column rather than across chips. Every time scales with the clock: a board at 252 MHz takes 1.5 times as long as the same board at 378 MHz. Clock for clock, the RP2350 needs a quarter to two-fifths fewer cycles per statement than the RP2040. At the same clock it would be 1.3 to 1.7 times as fast.

Each test ran three times and kept its best time; repeated runs agree within 1-2%. A difference of less than about 10% between two *different* tests should not be relied on, though. Changing a program moves where its parts land in the processor's flash cache. On the RP2040 that alone moved single tests by up to 13% between two versions of the test program. When this guide calls two forms "the same", that is what it means.

2. Quick answers

Question	Answer
One statement per line, or several with colons?	No difference worth having. Write whichever reads better.
FOR ... NEXT or DO ... LOOP?	FOR ... NEXT, by far. An empty FOR pass takes 2.2 µs on the RP2040. The best DO loop takes 5 to 6 µs, a DO whose condition names a variable 11 µs, and a GOTO loop 20 µs. Compiled: FOR 1.1 µs, DO 2.2 µs. Use an integer loop variable.
`NEXT` or `NEXT x`?	The same. NEXT does not look the name up; it is only a little more text to step over (about 2%).
Is a FOR loop's limit worked out on every pass?	No. The start, limit and STEP are worked out once, when the FOR runs. A DO WHILE or LOOP UNTIL condition *is* worked out on every pass, so work out its fixed parts before the loop.
Numbers, constants or variables?	A small whole number costs about the same as a variable or a CONST. A number with a decimal point or an exponent costs more each time it is read: up to 6 µs more on the RP2040. Compiled, they all cost the same.
Is `CALL "sub"` faster than calling the SUB?	No. A direct call is faster: 32 µs against 40 µs on the RP2040, and 12 µs against 29 µs compiled. Beta 7 made CALL by name faster than it was, not faster than a direct call.
Do long names slow a program down?	Not with symbols, which are on by default, and not compiled. With `OPTION SYMBOLS OFF` each letter costs about 0.1 µs. A few commands and functions still read their arguments' names letter by letter (see section 6).

3. The questions in detail

3.1 One statement per line, or several?

```
a = b                   a = b : b = c : c = d
b = c
c = d
```

A statement is read and run the same way wherever it sits. Starting a new line costs the interpreter a few stores: it notes where the line is, for error messages and `TRACE`. A colon costs nothing to speak of.

Time for the three statements (µs):

	RP2040	RP2040 off	RP2350	RP2350 off	Compiled
three on three lines	14.1	20.9	17.9	19.4	2.27
the same on one line	14.5	21.4	18.4	19.9	2.18

The difference is within the noise, and it points different ways for the interpreter and the compiler. Write whichever reads better.

3.2 FOR ... NEXT or DO ... LOOP?

Use FOR ... NEXT wherever a loop counts. FOR works out its start, limit and STEP once. It keeps a pointer to the loop variable and to its NEXT, so each NEXT is one addition and one comparison, with no name to find and no expression to evaluate. A DO loop needs a separate statement to step its counter, and evaluates its condition on every pass. A loop made with GOTO is the slowest: `IF ... THEN GOTO` is a whole statement plus a jump.

Time per pass of an empty counting loop (µs):

	RP2040	RP2040 off	RP2350	RP2350 off	Compiled
`FOR i = 1 TO 20000 : NEXT` (float i)	2.16	2.16	2.28	2.28	1.12
the same with an integer, `i%`	1.43	1.43	2.00	2.00	0.86
`DO WHILE i < 20000 : INC i : LOOP`	5.77	6.92	6.10	6.35	2.20
`DO : INC i : LOOP UNTIL i >= 20000`	6.08	7.22	6.42	6.68	2.18
`DO : INC i : LOOP UNTIL i >= n`	10.9	14.3	11.4	12.2	2.17
`DO : i = i + 1 : LOOP UNTIL i >= 20000`	10.4	13.8	10.9	11.4	2.20
`DO WHILE i% < 20000 : INC i% : LOOP`	5.02	6.28	5.87	6.19	2.08
`label: INC i : IF i < 20000 THEN GOTO label`	19.9	23.3	19.7	21.2	3.74

(The loops are written on one line here to save space; in the tests each statement had its own line.)

- **An integer loop variable makes FOR faster:** 1.43 instead of 2.16 µs on the RP2040, which does its floating point in software, and 0.86 instead of 1.12 µs compiled.
- **The interpreter has a fast path for DO conditions.** A condition made of a variable, a comparison and a plain number is set up once and then checked directly. Examples are `i < 20000`, `LOOP UNTIL x >= 4.0` and `20000 > i`. Any other condition is evaluated in full on every pass, which takes nearly twice as long:

one compared with a variable or a CONST (`i >= n`), one using `AND`, or one using an array element. So if a DO loop's limit is fixed, write it as a number.
- **`INC i` is quicker than `i = i + 1`** (see 4.2).
- **Compiled, every form of DO costs the same,** about 2.2 µs a pass, and FOR is still twice as fast.
- **Use DO when you do not know how many passes there will be:** waiting for a key or a sensor, or repeating until a result settles. There the time goes on the loop's body.

3.3 NEXT or NEXT x?

NEXT does not look its variable up. It finds its FOR from where it is in the program, the same way with or without a name, so the name only adds a little text to step over.

Time per inner pass of two nested FOR loops (µs):

	RP2040	RP2040 off	RP2350	RP2350 off	Compiled
`NEXT : NEXT`	2.30	2.31	2.44	2.44	1.13
`NEXT k : NEXT j`	2.35	2.36	2.51	2.51	1.13
`NEXT k, j`	2.38	2.39	2.56	2.56	1.13

`NEXT` alone is about 2% faster interpreted, and the same compiled. Naming the variable makes long nested loops easier to read, and that is worth far more than 2%. `NEXT k, j` closes both loops in one statement but is no faster than two NEXTs.

3.4 Is a FOR loop's limit worked out on every pass?

No. FOR evaluates the start value, the limit and STEP once, when the FOR statement runs, and keeps them. Changing a variable used in the limit or STEP inside the loop does not change the number of passes, interpreted or compiled:

```
n = 5
FOR i = 1 TO n
  n = 2            ' does not shorten the loop
NEXT               ' runs 5 times
```

A DO loop is the opposite: its WHILE or UNTIL condition is evaluated on every pass. Work out anything in it that does not change before the loop:

Time per pass (µs):

	RP2040	RP2040 off	RP2350	RP2350 off	Compiled
`FOR i = 1 TO n`	2.16	2.16	2.28	2.28	1.12
`FOR i = 1 TO a1 + b1 + c1 + SQR(x1)`	2.17	2.17	2.28	2.28	1.12
`DO WHILE i < a1 + b1 + c1 + SQR(x1)`	24.9	32.3	24.0	25.9	3.46
`m = a1 + b1 + c1 + SQR(x1)`, then `DO WHILE i < m`	11.4	14.8	11.3	12.0	2.19

The two FOR rows are identical: the long limit costs nothing per pass. In the DO loop the same expression doubles the time of every pass, on every board.

3.5 Numbers, constants or variables?

A number in your program is kept as its digits and converted every time the statement runs. A CONST is a variable that cannot be changed, and is found through its symbol like any other variable.

Time of the statement (µs):

	RP2040	RP2040 off	RP2350	RP2350 off	Compiled
`a = b + 10`	8.12	10.4	8.35	8.87	1.01
`a = b + K10` (a CONST)	8.52	12.2	9.26	10.2	1.17
`a = b + v10` (a variable)	8.06	11.8	9.09	9.99	1.00
`a = b * 3.14159265`	11.7	14.0	10.3	10.8	1.07
`a = b * KPI` (a CONST)	8.28	11.9	9.22	10.1	1.07
`a = b * vpi` (a variable)	8.23	11.9	9.16	10.1	1.07
`a = b * PI`	7.57	9.85	8.19	8.71	1.07
`a = b * 1.5E-3`	14.4	16.6	11.4	11.9	1.07

- **A small whole number is as quick as a variable or a CONST:** converting `10` takes a couple of multiplications.
- **A number with a decimal point costs more,** most of all on the RP2040, which has no floating-point hardware. `3.14159265` costs 3.4 µs more than a CONST, and `1.5E-3` 6 µs more. In a busy statement, a CONST or a variable is quicker than a decimal number. `PI` is the quickest of all.
- **With `OPTION SYMBOLS OFF` the order changes.** Every name is then found by its spelling, so a number in the text is quicker than a CONST or a variable.
- **Compiled, they are all the same.** The compiler converts each number once, when it compiles.

Arguments of graphics commands. The coordinates of `LINE`, `BOX`, `PIXEL` and the like must be whole numbers, so a float variable is converted every time the command runs:

Time of one `LINE` *of 91 pixels, drawing included (µs):*

	RP2040	RP2040 off	RP2350	RP2350 off	Compiled
`LINE 10, 10, 100, 100`	81.5	81.5	78.2	78.2	74.1
with four CONSTs	86.1	96.2	83.4	85.6	74.3
with four integer variables	85.9	95.4	82.9	85.0	74.1
with four float variables	98.4	108	89.1	91.2	80.5

Drawing the line takes most of the time. Of what is left, literal numbers are quickest. Integer variables and CONSTs add 4 to 5 µs for the four arguments (a CONST made from a whole number is an integer). Float variables add 11 to 17 µs, and they are the only kind that still costs anything compiled. Keep coordinates in integer variables (`x%`, or `DIM INTEGER`).

3.6 Is CALL "sub" faster than calling the SUB?

No: calling the SUB by name is faster. `CALL` takes the name as a string expression, looks it up by its spelling, and then makes the call as a direct call does. Beta 7 made CALL faster than it had been, because it no longer spells out the rest of the statement before each call. That made it faster than before, not faster than a direct call.

Time of the call (µs). The SUB `AddIt(p1, p2)` *and the FUNCTION* `FAdd(p1, p2)` *each contain one statement,* `a = p1 + p2`*:*

	RP2040	RP2040 off	RP2350	RP2350 off	Compiled
`a = b + c` written in place	8.06	11.5	9.08	9.83	1.00
a SUB with no parameters, empty	11.3	12.6	8.10	9.35	3.85
`GOSUB` to `a = b + c : RETURN`	11.8	16.8	14.8	16.8	5.30
`AddIt b, c`	32.1	47.7	28.5	34.5	11.8
`CALL "AddIt", b, c`	39.7	53.0	36.3	41.1	28.8
`CALL f$, b, c`	38.5	53.0	35.3	40.3	27.9
`a = FAdd(b, c)`	42.6	60.4	38.1	45.4	17.9
`a = CALL("FAdd", b, c)`	62.5	80.4	47.6	53.6	39.8
a SUB with `LOCAL la, lb, lc`, otherwise empty	30.3	37.5	24.9	30.0	13.1

CALL by name costs 7 to 8 µs more per call interpreted. Compiled it costs two and a half times a direct call, because CALL itself is not compiled. Use it when the SUB to call is chosen while the program runs, from a table of names: BASIC's answer to a function pointer. Everywhere else, call the SUB directly. The same goes for `CALL(` and FUNCTIONs.

Calls are the most expensive thing in MMBasic. On the RP2040 a SUB with two parameters costs four times its one-statement body; compiled it costs twelve times. Each parameter and each LOCAL is a variable made when the SUB starts and removed when it ends. Three LOCALs add about 19 µs to a call on the RP2040, and 9 µs compiled. So:

- **Do not break a tight inner loop up into tiny SUBs.** Put the loop inside the SUB rather than the SUB inside the loop.
- **In a small SUB that is called very often, each parameter and each LOCAL is a large part of the cost:** about 6 µs each on the RP2040 and 3 µs compiled. A LOCAL earns its place in a bigger SUB, or a recursive one.
- **GOSUB and RETURN cost less than a SUB call interpreted:** about 4 µs on the RP2040, against 11 µs for a SUB with no parameters. They take no parameters and have no LOCALs, though, and they run on the interpreter even when the program is compiled, where a SUB call costs about the same.

3.7 Do long names slow a program down?

Not in V7 with symbols, which are on by default. A saved program stores every name as a symbol of two or three bytes, whatever the length of the name. When the program runs, the interpreter remembers where each symbol's variable, SUB or label is, so `TheFirstVariable` costs the same as `a`. Compiled code does not see names at all.

Time of the statement (µs):

	RP2040	RP2040 off	RP2350	RP2350 off	Compiled
`a = b + c`	8.06	11.5	9.08	9.83	1.00
the same with names of 16 or 17 letters	8.15	16.7	9.19	14.0	1.00
the same with names of 30 letters	8.16	21.1	9.19	17.6	1.00
`AddIt b, c`	32.1	47.7	28.5	34.5	11.8
`AddTwoNumbersAndStoreTheResult b, c`	32.6	52.9	28.6	37.7	11.8
`a = RGB(rr, gg, bb)`	20.1	21.3	18.2	18.5	7.15
the same with names of 26 to 28 letters	52.1	53.3	38.6	38.9	7.15

Long names cost time in three cases:

1. **With `OPTION SYMBOLS OFF`.** Each letter of each name costs about 0.11 µs on the RP2040 at 378 MHz, and 0.09 µs on the RP2350 at 252 MHz. Three 30-letter names made the statement 84% slower on the RP2040.
2. **At the command prompt, and in strings run by `EXECUTE` or evaluated by `EVAL`.** These are never stored as symbols.
3. **In the arguments of the commands and functions that still read names by their spelling** (section 6). `RGB(` is one of them: with symbols on or off, three 27-letter names inside it took 32 µs longer than three two-letter names. Compiled, `RGB(` is compiled and the names cost nothing.

So use names that make the program clear. Keep them short only in the arguments of the commands and functions listed in section 6, and then only in a busy loop.

4. Other things that make a difference

4.1 Integers and floats

Time of the statement (µs):

	RP2040	RP2040 off	RP2350	RP2350 off	Compiled
`a = b + c`	8.06	11.5	9.08	9.83	1.00
`ia% = ib% + ic%`	7.71	11.5	9.05	10.0	0.99
`a = b * c`	8.24	11.7	9.16	9.93	1.07
`ia% = ib% * ic%`	7.77	11.5	9.09	10.0	0.93
`a = b / c`	8.46	11.9	9.35	10.1	1.57
`ia% = ib% \ ic%`	7.80	12.5	9.37	10.3	1.51
`a = b * b`	8.23	11.7	9.16	9.92	1.07
`a = b ^ 2`	8.62	12.3	8.86	9.35	2.05
`a = SQR(b)`	8.42	10.7	9.60	10.1	1.30
`a = SIN(b)`	12.2	14.5	10.5	11.0	1.88

The arithmetic is lost in the reading: integer and float statements take the same time, within 8%. Integers do help in a few places:

- **loop variables:** FOR with an integer variable is a third faster on the RP2040 (section 3.2);
- **coordinates,** and anything else a command needs as a whole number (section 3.5);
- **`LOCAL INTEGER`** variables in a SUB: a loop on them ran 11% faster than on float LOCALs on the RP2040 (section 4.7).

Two more points about the maths:

- **`^`:** compiled, `b ^ 2` takes twice as long as `b * b`, and `^` with an integer on its left is not compiled at all. Interpreted, the two cost the same.
- **`SIN` and other maths functions:** on the RP2040, which has no floating-point hardware, `SIN` costs 4 µs more than `SQR`; on the RP2350, 1 µs.

4.2 INC

`INC a` is more than twice as fast as `a = a + 1` interpreted. Compiled, the two are the same. `INC a, n` adds n.

	RP2040	RP2040 off	RP2350	RP2350 off	Compiled
`INC a`	3.43	4.57	3.68	3.93	0.98
`a = a + 1`	7.78	10.1	8.11	8.62	1.01
`INC ia%`	3.42	4.68	3.73	4.05	0.91
`ia% = ia% + 1`	6.97	9.49	7.93	8.57	0.91

4.3 Arrays

Time of the statement (µs):

	RP2040	RP2040 off	RP2350	RP2350 off	Compiled
`a = b + c`	8.06	11.5	9.08	9.83	1.00
`a = x(5) + y(5)`	13.9	18.9	15.0	16.5	1.77
`a = x(q) + y(q)`	19.8	27.1	19.2	21.2	2.90
`a = m2(5, 5) + m2(6, 6)`	18.4	24.0	19.7	21.6	2.59

An element costs more than a simple variable: its index has to be evaluated and checked, and its address worked out. On the RP2040, reading two elements instead of two simple variables added 6 µs with fixed indexes and 12 µs with an index variable; compiled, 0.8 µs and 1.9 µs. If a busy loop uses the same element three or more times, copy it into a simple variable once.

4.4 IF, ELSEIF and SELECT CASE

Time of the statement or block (µs):

	RP2040	RP2040 off	RP2350	RP2350 off	Compiled
`IF b < c THEN a = 1`	13.5	15.8	14.3	14.8	1.03
`IF b < c THEN` / `a = 1` / `ENDIF`	10.8	13.0	11.3	11.8	1.03
`IF` and seven `ELSEIF`s, the eighth test true	72.6	85.1	68.8	71.6	6.21
`SELECT CASE` with eight `CASE`s, the eighth true	29.5	34.7	30.5	31.1	6.56

- **A multi-line IF is quicker than a single-line one** interpreted: 10.8 µs against 13.5 µs. When a program is made ready to run, MMBasic indexes its multi-line IF blocks, so a false test jumps straight to the next ELSEIF, ELSE or ENDIF. Compiled, the two are the same.
- **A chain of ELSEIFs is slow to reach its later arms:** it tests each condition in turn, each a full statement. SELECT CASE evaluates its selector once and is two and a half times faster here. Compiled, both take about 6 µs. In either form, put the most frequent case first.
- **MMBasic always evaluates both sides of `AND` and `OR`:** they are bitwise operators, not short-circuit tests. If one test is cheap and usually decides the result, nest two IFs so that the expensive test runs only when it is needed.

4.5 Comments, REM and spaces

Time of the statement, with whatever is around it (µs):

	RP2040	RP2040 off	RP2350	RP2350 off	Compiled
`a = b`	4.69	6.98	5.96	6.46	0.76
`a = b ' a 40-character comment`	6.13	8.42	7.60	8.11	0.76
a `'` comment line, then `a = b`	5.05	7.34	6.36	6.87	0.80
a `REM` line, then `a = b`	6.88	9.17	8.94	9.44	1.37
`a = b + c`	8.06	11.5	9.08	9.83	1.00
`a=b+c`	7.41	10.8	8.43	9.20	1.00

- **A comment line costs almost nothing:** the interpreter jumps straight to the next line. Compiled it costs nothing at all.
- **A comment at the end of a statement is read through, every time the statement runs.** In the test it added 1.4 µs, nearly a third of a short statement. Compiled it costs nothing. In a busy loop, put comments on lines of their own.
- **`REM` is a command, not a comment line.** It costs more than a `'` comment line, and it is the one kind of comment that still costs time compiled. Use `'`.
- **Spaces cost a little:** `a=b+c` is 8% faster than `a = b + c` interpreted, and the same compiled. That is not worth an unreadable program. `LOAD "prog", C` (also `AUTOSAVE C` and `XMODEM C`) removes comments, blank lines and unneeded spaces as the program is loaded. The file on disk is not changed. `LIST` and `EDIT` then show the crunched program, and error messages give its line numbers. Compiled programs gain nothing from it.

4.6 Strings

Time of the statement (µs):

	RP2040	RP2040 off	RP2350	RP2350 off	Compiled
`s$ = t$ + u$`	9.67	13.0	11.1	11.9	4.08
`s$ = LEFT$(t$, 3)`	13.8	16.0	14.2	14.8	7.90
`s$ = STR$(b)`	35.1	53.6	21.7	22.3	18.1

Joining two strings costs about what adding two numbers does. Turning a number into text is expensive, and compiling hardly helps, because the time goes on formatting the number: `STR$` of a float takes 35 µs on the RP2040. `PRINT` of a number and `FORMAT$` do the same work. In a loop, convert a number to text only when its value has changed.

4.7 LOCAL or global variables

Time per pass of `FOR ... : a = b + c : NEXT` *(µs):*

	RP2040	RP2040 off	RP2350	RP2350 off	Compiled
at the top level of the program	10.2	13.6	11.4	12.1	2.12
in a SUB, on global variables	10.3	14.1	11.5	12.4	2.17
in a SUB, on LOCAL variables	10.2	14.0	11.3	12.8	2.17
in a SUB, on LOCAL INTEGER variables	9.07	12.9	10.8	12.4	1.82

In V7 a local variable is found as quickly as a global one. Use locals freely inside a SUB. What LOCAL costs is paid when the SUB starts (section 3.6), not when the variables are used.

4.8 Working something out once

Moving a calculation that does not change out of a loop always pays (section 3.4). Within one statement it pays only when the repeated part costs more than an extra statement:

Time (µs):

	RP2040	RP2040 off	RP2350	RP2350 off	Compiled
`a = SIN(b) * c + SIN(b) * d`	29.4	35.1	23.7	25.0	3.81
`sx = SIN(b) : a = sx * c + sx * d`	27.3	35.6	26.3	28.3	3.45

On the RP2040, keeping `SIN(b)` in a variable saved 2 µs. On the RP2350 interpreted it *cost* 2.6 µs, because the extra statement costs more than a `SIN` there. So:

- always move fixed work out of loops;
- within a statement, store only what is expensive: a user FUNCTION, a string function, or a maths function on the RP2040.

4.9 Outside your program

- **The clock.** Every time in this guide scales with the CPU clock. `OPTION CPUSPEED` sets it on the builds without video. On the VGA and HDMI builds the clock follows `OPTION RESOLUTION`. `MM.INFO(CPUSPEED)` reports it.
- **The build.** At the same clock, the plain PicoMite builds are quicker than those that also run WiFi, a USB host or an HDMI display, by a few per cent up to about a fifth. Those builds do some housekeeping between statements; section 4.10 measures what the wireless builds' housekeeping costs and how to switch it off.
- **The console.** `PRINT` to a screen that has to scroll is slow, because every new line moves the whole screen. Print less inside loops, or send the output to the serial console with `OPTION CONSOLE SERIAL`.
- **Interrupts.** An interrupt that is set up but not firing (a `SETTICK`, a pin, a COM port) costs almost nothing. Its routine is BASIC like any other, though, so keep it short.

4.10 The wireless builds: what runs between statements

The WiFi builds and the Bluetooth host builds (WebMite, PicoMiteHDMIWEB, PicoMiteRP2350BTH, PicoMiteHDMIBTH and PicoMitePC3) service their radio from the statement loop. Once a millisecond, or once every hundred statements if those are quicker, the interpreter polls the wireless chip, runs the Bluetooth stack's timers, gives the WiFi stack its turn when a network is joined, and feeds a Bluetooth speaker when one is playing. Nothing of this is tied to your program, so the cost is a slice of every millisecond: a long loop of short statements shows it most, a program that spends its time in a `MATH` command or a `PAUSE` hardly notices it.

The table measures that slice on one board, a Pico Computer 3 at 315 MHz running the PicoMitePC3 build with `OPTION COMPILE OFF`, which can switch each half of its radio off with `OPTION RADIO`. The anchor is solar_eclipse, a 70 KB interpreted program; the empty loop is `FOR i = 1 TO 20000 ... NEXT` with nothing inside, per pass. Runs repeat within 2%.

Running between statements	solar_eclipse	empty loop	cost
nothing: `OPTION RADIO NONE` (the PicoMiteHDMIUSB's speed)	14.15 s	1.21 µs	-
the WiFi stack, a network joined (`OPTION RADIO WEB`)	14.57 s	1.53 µs	+3%
the Bluetooth host, nothing connected (`OPTION RADIO BLUETOOTH`)	15.11 s	1.45 µs	+7%
both, audio on the I2S DAC (`OPTION RADIO BOTH`)	15.04 s	1.68 µs	+6%
both, `OPTION AUDIO DISABLE`	15.07 s	1.78 µs	+7%
both, a Bluetooth speaker connected and silent	15.06 s	1.76 µs	+6%
both, the speaker playing	18.04 s	2.04 µs	+27%

What the rows say:

- **WiFi costs about 3% while a network is joined**, and nothing while it is not: the stack has nothing to do until there are packets. A busy connection (a TFTP transfer, a web page being served) costs more for as long as it lasts.
- **The Bluetooth host costs about 7% with nothing connected.** Most of that is the scan for a keyboard, which runs whenever no BLE keyboard is connected, and the stack's own housekeeping. It is the same on the PicoMiteRP2350BTH and PicoMiteHDMIBTH, which cannot switch it off.
- **An audio output that is not playing costs nothing**, whether it is the I2S DAC, nothing at all, or a Bluetooth speaker sitting connected and silent. Before V7 RC1 a configured speaker cost a third of the interpreter's time even when silent; it no longer does.
- **A Bluetooth speaker that is playing costs about a quarter.** The SBC encoder takes a tenth of the processor by itself, and the rest is the packets going out through the radio. The I2S DAC plays the same files through a DMA buffer and costs nothing you can measure. The cost stops the moment the sound stops.
- **`SAVE` and other flash writes pause a playing speaker** for as long as they take, because a flash write runs with interrupts off and the stack is not polled meanwhile: a 250 KB `SAVE` silences the speaker for about five seconds. The stream carries on afterwards.

So on a PicoMitePC3 that is used as an HDMI computer, `OPTION RADIO NONE` gives it the plain PicoMiteHDMIUSB's speed; a WebMite-style use is `OPTION RADIO WEB`; and with both radios in use the only cost worth planning around is a speaker that is playing while the program runs.

5. Let the firmware do the loop: MATH, ARRAY, SORT, MEMORY and the graphics array forms

A BASIC loop over an array costs a statement or two per element: 12 to 25 µs on the RP2040, 3 to 5 µs compiled. The `MATH`, `ARRAY`, `SORT` and `MEMORY` commands, and the array forms of the drawing commands, run the same loop in C. That takes from 0.03 µs to a few microseconds per element. Even compiled BASIC is 10 to 30 times slower than these commands.

Time per element, arrays of 1000 floats (µs):

	RP2040	RP2350	Compiled
`b(j) = a(j) * 2.5` in a loop	21.7	21.1	4.16
`MATH SCALE a(), 2.5, b()`	0.60	0.22	0.22
`b(j) = a(j) + 7` in a loop	20.0	20.3	4.08
`MATH ADD a(), 7, b()`	0.84	0.24	0.23
`c(j) = a(j) * b(j)` in a loop	25.8	26.5	5.11
`MATH C_MUL a(), b(), c()`	0.59	0.20	0.20
`b(j) = a(j)` in a loop	17.3	18.4	3.85
`b() = a()`	0.03	0.03	0.03
`MEMORY COPY FLOAT`	0.06	0.06	0.05
`arr(j) = 1` in a loop	12.2	12.4	2.91
`MATH SET 1, arr()`	0.54	0.15	0.15
sum: `tot = tot + a(j)` in a loop	15.7	16.7	3.07
`MATH(SUM a())` or `MATH(MEAN a())`	0.36	0.12	0.12
largest value and its index: an IF in a loop	23.6	24.6	3.13
`mx = MATH(MAX a(), ix%)`	0.30	0.15	0.14
dot product in a loop	24.3	24.8	4.34
`MATH(DOTPRODUCT a(), b())`	0.86	0.26	0.25
rotate points about a centre in a loop	86.5	79.1	12.1
`MATH V_ROTATE 160, 120, 0.3, xi(), yi(), xo(), yo()`	4.50	1.28	1.27
scale to 0-239: find the minimum and maximum, then scale, in loops	67.4	69.7	9.47
`MATH WINDOW a(), 0, 239, b()`	2.98	0.95	0.94
limit to 100-900 with two IFs in a loop	52.8	54.9	6.68
`MATH CLAMP a(), 100, 900, b()`	0.53	0.23	0.22
`PIXEL px%(j), py%(j)` in a loop (per pixel)	28.3	28.5	9.11
`PIXEL px%(), py%()` (per pixel)	0.28	0.32	0.30

(The RP2350 column is the interpreter; the commands themselves are not compiled, so they take the same time compiled.)

When it pays

A MATH command has a fixed cost of its own, about 13 to 16 µs, because its arguments are read as text. After that, each element costs very little. Here is the sum of a small array, repeated:

Time per sum (µs), including about 2 µs (1.1 µs compiled) for the loop that repeats it:

	RP2040	RP2350	Compiled
`tot = s4(0) + s4(1) + s4(2) + s4(3)`	28.2	29.2	4.11
4 elements in a FOR loop	81.7	90.8	16.4
4 elements: `MATH(SUM s4())`	18.4	15.8	13.9
16 elements in a FOR loop	271	295	52.9
16 elements: `MATH(SUM s16())`	22.8	17.3	15.3
64 elements in a FOR loop	1023	1099	200
64 elements: `MATH(SUM s64())`	39.0	21.9	19.9

- **Interpreted, a MATH command wins from the smallest arrays:** for four elements it beats the loop, and even beats writing the four additions out.
- **Compiled, writing a handful of elements out is quickest.** The MATH command still beats a loop from four elements, and at 64 elements it is ten times faster.
- **The commands work on whole arrays.** Size your arrays to the data they hold, or take out the part you need first with `ARRAY SLICE`. A loop that changes only a few elements gains nothing.

What to use them for

The manual describes each command in full. These are the jobs they do well:

- **Clearing, filling and copying.** `ARRAY SET value, a()` (also written `MATH SET`) is the fastest way to clear an array, and works for strings too. `b() = a()` copies a whole array. `MATH SCALE a(), 1, b()` copies between integer and float arrays. `MEMORY COPY` and `MEMORY SET` work on raw memory and byte buffers.
- **Rows and columns of multi-dimensional arrays.** `ARRAY SLICE` copies one row or column out into a one-dimensional array, and `ARRAY INSERT` puts one back. They suit game boards, tile maps and tables.
- **The same operation on every element:**
  - `MATH ADD` adds a value; `MATH SCALE` multiplies by one.
  - `MATH C_ADD`, `C_SUB`, `C_MUL` and `C_DIV` work element by element between two arrays.
  - `MATH C_AND`, `C_OR` and `C_XOR` combine bit masks; `MATH SHIFT` shifts the bits of integer arrays.
  - `MATH POWER` raises every element to a power.
  - `MATH CLAMP` keeps values within limits.
  - `MATH INTERPOLATE` blends two arrays, for smoothing or for animation between two positions.
- **Statistics and searching:**
  - `MATH(SUM`, `MATH(MEAN`, `MATH(MEDIAN` and `MATH(SD` summarise an array.
  - `MATH(MAX` and `MATH(MIN` can also return where the value is, to find the nearest, farthest or best.
  - `MATH(CROSSING` finds where captured data first crosses a level, ignoring short spikes.
  - `MATH(DOTPRODUCT`, `MATH(MAGNITUDE`, `MATH(CORREL` and `MATH(CHI` cover vectors and correlation.
- **Plotting data.** `MATH WINDOW` scales a set of readings to the screen's range and also returns their minimum and maximum. Then `LINE GRAPH x(), y(), colour`, `LINE PLOT`, or the array forms of `PIXEL` and `LINE` draw the whole set in one command.
- **Geometry and 3-D:**

  - `MATH V_ROTATE` rotates a whole set of points, such as a polygon, a vector sprite or a star field.
  - `POLYGON` draws many polygons from arrays.
  - `MATH M_MULT` and `MATH V_MULT` multiply matrices and vectors, for 2-D and 3-D transforms; `MATH M_INVERSE` and `M_TRANSPOSE` complete the set.
  - The quaternion commands (`MATH Q_CREATE`, `Q_EULER`, `Q_ROTATE` and the rest) handle 3-D orientation, and `DRAW3D` builds on them.
- **Signals.** `MATH FFT` (with `FFT MAGNITUDE`, `FFT PHASE` and `FFT INVERSE`) analyses a captured waveform. `MATH SINC` filters or resamples one.
- **Drawing many things at once.** `PIXEL`, `LINE` and `BOX` take arrays for any of their arguments and draw one item per element. 1000 pixels from arrays take 0.3 ms; drawn one at a time in a loop they take 28 ms.
- **Sorting.** `SORT a()` sorts numbers or strings. With an index array, it also tells you where each element came from, so that related arrays can be put into the same order.

A note on SORT. `SORT` uses a simple exchange sort, so its time grows with the square of the number of elements. Twice as many elements take four times as long:

Sorting	RP2040	RP2350	Compiled
1000 numbers with `SORT`	134 ms	66 ms	66 ms
1000 numbers, a Shell sort written in BASIC	764 ms	840 ms	118 ms
4000 numbers with `SORT`	2156 ms	1066 ms	1066 ms
4000 numbers, a Shell sort written in BASIC	3632 ms	-	549 ms

`SORT` itself is not compiled, so it takes the same time either way. Interpreted, `SORT` is quicker than a BASIC sort up to several thousand elements: at 4000 on the RP2040 it took 2.2 s against 3.6 s. Compiled on an RP2350, a Shell sort written in BASIC overtakes it at around 2000 elements, and at 4000 it takes half the time.

6. Symbols: on (the default) and OPTION SYMBOLS OFF

When a program is saved, V7 stores every variable, constant, SUB, FUNCTION and label name as a symbol of two or three bytes. When the program runs, MMBasic builds tables that say where each symbol's variable, SUB or label is. Each name is looked up once, the first time it is used; from then on it is found directly. `LIST` and `EDIT` show every name exactly as you typed it, and a program saved to a file is plain text as before.

What symbols are worth. Without them, every name is found by its spelling, as in V6:

	RP2040	RP2040 off	RP2350	RP2350 off
`a = b + c`	8.06	11.5	9.08	9.83
`AddIt b, c` (a SUB with two parameters)	32.1	47.7	28.5	34.5
`a = FAdd(b, c)`	42.6	60.4	38.1	45.4
`GOSUB` to `a = b + c : RETURN`	11.8	16.8	14.8	16.8

Without symbols, statements are 25 to 50% slower on the RP2040 and 5 to 20% slower on the RP2350, and long names cost more again (section 3.7). On an RP2350 a program with `OPTION SYMBOLS OFF` is also never compiled.

When to use `OPTION SYMBOLS OFF`. Only when a program runs out of memory. The symbol tables take about 20 bytes for each name plus a fixed part, typically 3 to 5 KB. They share memory with the program's variables, strings and arrays (on an RP2350 with PSRAM, most of them go into PSRAM). You do not have to guess: when a program runs out of memory for a request no larger than the tables, the error says so:

```
Error : Not enough memory for 1920 bytes: put OPTION SYMBOLS OFF first
```

Put `OPTION SYMBOLS OFF` on a line of its own at the top of the program. Only blank lines, comment lines and a `LIBRARY LOAD` may come before it. It affects only that program.

Where names are still found by their spelling, with symbols on:

- commands typed at the command prompt, and strings run by `EXECUTE` or evaluated by `EVAL`;
- the name in `CALL name$` and `CALL(name$, ...)` (section 3.6);
- in V7.0.00b7, the arguments of the commands and functions that have not yet been converted to read symbols. A command's whole statement, or a function's arguments, are spelled out in full before they run, and the names in them are looked up by their spelling.
  - **Commands that read symbols:** assignments; `IF`, `ELSEIF`, `ELSE`, `ENDIF`; `FOR`, `NEXT`, `DO`, `LOOP`, `EXIT`, `CONTINUE`; `SELECT CASE`, `CASE`, `END SELECT`; `SUB`, `FUNCTION`, `END SUB`, `END FUNCTION`, `RETURN`, `IRETURN`, `CALL`; calls to your own SUBs; `PRINT`, `INC`, `GOTO`, `GOSUB`, `DIM`, `LOCAL`, `STATIC`, `CONST`, `DATA`, `REM`; and the drawing commands `PIXEL`, `LINE`, `BOX`, `RBOX`, `CIRCLE`, `TRIANGLE`, `ARC`, `BEZIER`, `CLS`, `COLOUR`, `TEXT`, `FONT`, `BLIT`, `SPRITE` and `REFRESH`. Every other command is spelled out: for example `MATH`, `MEMORY`, `POKE`, `PAUSE`, `PLAY`, `SETPIN`, `POLYGON`, `SORT`, `TILEMAP` and `FRAMEBUFFER`.
  - **Functions that read symbols:** `ABS`, `ACOS`, `ASC`, `ASIN`, `ATAN2`, `ATN`, `BASE$`, `BIN$`, `BIN2STR$`, `BIT`, `BOUND`, `BYTE`, `CALL`, `CHOICE`, `CHR$`, `CINT`, `COS`, `DEG`, `EVAL`, `EXP`, `FIELD$`, `FIX`, `FLAG`, `HEX$`, `INSTR`, `INT`, `LCASE$`, `LEFT$`, `LEN`, `LOG`, `MAX`, `MID$`, `MIN`, `RAD`, `RIGHT$`, `RND`, `SCHANGE$`, `SGN`, `SIN`, `SPACE$`, `SQR`, `STR$`, `STR2BIN`, `STRING$`, `TAB`, `TAN`, `TOPBOTTOM`, `TRIM$`, `UCASE$` and `VAL`. Every other function is spelled out, among them `RGB(`, `PEEK(`, `PIN(`, `PORT(`, `PIXEL(`, `KEYDOWN(`, `MM.INFO(`, `MATH(`, `MAP(`, `FORMAT$(`, `DEVICE(`, `SPRITE(`, `TILEMAP(` and `TOUCH(`.

The cost is the same as with `OPTION SYMBOLS OFF`: a few microseconds per name, plus about 0.1 µs per letter. It only matters inside a busy loop, and there it is easily avoided: keep the names short in those statements, or work the value out into a variable first. On an RP2350, compiling removes the cost for those of them that compile: `RGB(`, `PIXEL(`, `KEYDOWN(`, `MAP(`, the common forms of `PEEK(` and most uses of `MM.INFO(`.

7. The compiler (RP2350): OPTION COMPILE

On the RP2350 builds, `OPTION COMPILE ON` (typed at the command prompt) makes the next `RUN` compile the program into native operations, held apart from the program itself. Statements the compiler cannot take stay as references to your text and run on the interpreter, mixed in with the compiled ones. Nothing about the program changes: `LIST`, `EDIT`, error messages and line numbers, `TRACE`, `ON ERROR`, interrupts and `CHAIN` all behave as before. The option is saved, so it stays on until you type `OPTION COMPILE OFF`. `OPTION COMPILE SHADOW` runs compiled and checks every compiled value against the interpreter, stopping at the first line where they differ. The release notes describe all this in full.

What it gains. These are the same statements as in the earlier sections, on the same RP2350:

	interpreted	compiled	faster by
`a = b + c`	9.08	1.00	9.1x
`IF b < c THEN a = 1`	14.3	1.03	14x
`a = x(5) + y(5)`	15.0	1.77	8.5x
`s$ = t$ + u$`	11.1	4.08	2.7x
a FOR ... NEXT pass	2.28	1.12	2.0x
`DO : INC i : LOOP UNTIL i >= n`, per pass	11.4	2.17	5.3x
`AddIt b, c` (a SUB with two parameters)	28.5	11.8	2.4x

	interpreted	compiled	faster by
`a = FAdd(b, c)`	38.1	17.9	2.1x
`CALL "AddIt", b, c` (not compiled)	36.3	28.8	1.3x
`s$ = STR$(b)`	21.7	18.1	1.2x
`PIXEL px%(j), py%(j)` in a loop, per pixel	28.5	9.11	3.1x

Whole programs gain 1.4 to 5 times: the release notes list some. The gain is smaller where the time goes on work the compiler cannot speed up: the maths library, drawing, strings, and statements that are not compiled.

Seeing how much of a program runs compiled. After a run, `PRINT MM.INFO(COMPILE)` shows `RAN` (statements run) and `CODE` (how many of those ran compiled). `CODE` divided by `RAN` is the share done compiled. To see what ran as text, put `OPTION PROFILING ON` as the program's first line and end the program with `END`. The report then lists, busiest first, the commands that ran on the interpreter. Add `, SAMPLE` to also get the busiest lines.

Writing for the compiler. Most programs compile as they are. In the loops where a program spends its time, these are the things that still run as text, and what to use instead:

Runs as text	Instead
`GOTO`, `GOSUB`, `ON ... GOTO`	FOR, DO, block IFs and SUBs
`IF ... THEN GOTO`; `IF ... THEN line-number`; a single-line IF whose ELSE part is not an assignment	a block IF
`CALL name$`, `CALL(`	a direct call, where the target is known
`REM`	a `'` comment line
`PRINT`, `INPUT`, file and pin commands, `PAUSE`, `MATH`, `SPRITE`, `POLYGON`, `CLS`, `DIM`, `CONST`, `READ`, `ERASE`	keep them out of the innermost loop where you can; MATH commands are fast anyway (section 5)
`CONTINUE DO`	an IF around the rest of the loop's body (`EXIT DO` and `CONTINUE FOR` are compiled)
string arrays, structure members, FUNCTIONs that return a string, `SELECT CASE` on a string	numbers or simple string variables in the hot path
a user FUNCTION call in a FOR's start, limit or STEP, or in a DO condition	work it out into a variable before the loop
`^` with an integer on its left	a float, or `x * x`
`VAL` anywhere but alone on the right of an assignment	`n = VAL(s$)` first
the first run of a statement that creates a variable	DIM the variable before the loop (this costs only once)

Calls are what remains expensive in compiled code. A SUB call costs about 12 µs against 1 µs for a simple statement, so a tiny SUB called in an inner loop is worth writing in place.

8. Measuring your own program

Time a statement like this:

```
DIM FLOAT t, best, rep, i
best = 1E9
FOR rep = 1 TO 3
  t = TIMER
  FOR i = 1 TO 20000
    ' the statement to time
  NEXT
  t = TIMER - t
  IF t < best THEN best = t
NEXT rep
PRINT best * 1000 / 20000; " us per pass"
```

- `TIMER` counts milliseconds to the microsecond.
- Time the empty loop as well and subtract it.
- Keep the best of several runs: USB, the display and WiFi interrupt the program now and then.
- Declare the variables before the loop, so that their creation is not timed.
- Compare like with like: the same board, the same firmware and the same clock.
- Do not read much into a difference of less than about 10%. Where code lands in the flash cache moves single tests by that much, especially on the RP2040.
- On the RP2350, `RND` reseeds itself from the hardware random number generator every 100 draws. A program whose work depends on `RND` does a different amount each run, so compare time per unit of work (per frame, per point drawn).
- On a board with a screen, `PRINT`ing while timing measures the scrolling too.

To find where a whole program spends its time, put `OPTION PROFILING ON` as its first line and make sure it finishes with `END`. The report counts every command and times every SUB and FUNCTION. `OPTION PROFILING ON, SAMPLE` also lists the busiest lines. *OPTION PROFILING and Built-in Optimisations* (option-profiling-cache.pdf) describes the report in full.

9. When BASIC is not fast enough: mmb2csub

When the profile shows one or two SUBs or FUNCTIONs taking most of the time, the **mmb2csub** tool can compile them to machine code. It puts each one back into the program as a CSUB, and your calls to it do not change. Loops, arithmetic and array work run 10 to 35 times faster than interpreted. Maths functions and string formatting gain 3 to 4 times. The same CSUB runs on the RP2040 and the RP2350. See mmb2csub.pdf.

Appendix A. All the measurements

Every test, as measured. Times are in microseconds per pass of the test's loop, **with the loop included**: subtract test 0, the empty loop, to get the time of a statement alone, as the tables above do. Tests 3a to 3c are per pass of the inner loop, and 14a to 14f per element or pixel.

perfguide.bas and perfguide_off.bas (µs per pass):

Test		RP2040	RP2040 off	RP2350	RP2350 off	Compiled
0	empty FOR loop (the loop's own cost)	2.16	2.16	2.28	2.28	1.12
1a	three assignments, one per line	16.2	23.1	20.2	21.7	3.39
1b	three assignments, one line	16.7	23.6	20.6	22.2	3.30
2a	FOR float / NEXT	2.16	2.16	2.28	2.28	1.12
2b	FOR integer / NEXT	1.43	1.43	2.00	2.00	0.86
2c	DO / INC / LOOP UNTIL i >= 20000	6.08	7.22	6.42	6.68	2.18
2d	DO / INC / LOOP UNTIL i >= n (a variable)	10.9	14.3	11.4	12.2	2.17
2e	DO WHILE i < 20000 / INC / LOOP	5.77	6.92	6.10	6.35	2.20
2f	DO / i = i + 1 / LOOP UNTIL	10.4	13.8	10.9	11.4	2.20
2g	integer DO WHILE / INC / LOOP	5.02	6.28	5.87	6.19	2.08
2h	GOTO loop (IF ... THEN GOTO)	19.9	23.3	19.7	21.2	3.74
3a	nested, NEXT : NEXT	2.30	2.31	2.44	2.44	1.13
3b	nested, NEXT k : NEXT j	2.35	2.36	2.51	2.51	1.13
3c	nested, NEXT k, j	2.38	2.39	2.56	2.56	1.13
4a	FOR limit a variable	2.16	2.16	2.28	2.28	1.12
4b	FOR limit a1 + b1 + c1 + Sqr(x1)	2.17	2.17	2.28	2.28	1.12
4c	DO WHILE i < a1 + b1 + c1 + Sqr(x1)	24.9	32.3	24.0	25.9	3.46
4d	the same limit worked out once before the DO	11.4	14.8	11.3	12.0	2.19
5a	a = b + 10	10.3	12.6	10.6	11.2	2.12
5b	a = b + K10 (CONST)	10.7	14.4	11.5	12.5	2.29
5c	a = b + v10 (variable)	10.2	13.9	11.4	12.3	2.12
5d	a = b * 3.14159265	13.9	16.2	12.5	13.0	2.19
5e	a = b * KPI (CONST)	10.4	14.1	11.5	12.4	2.18
5f	a = b * vpi (variable)	10.4	14.1	11.4	12.3	2.18
5g	a = b * Pi	9.73	12.0	10.5	11.0	2.19
5h	a = b * 1.5E-3	16.5	18.8	13.7	14.2	2.19
5i	Line 10, 10, 100, 100	81.5	81.5	78.2	78.2	74.1
5j	Line lx1, ly1, lx2, ly2 (variables)	98.4	108	89.1	91.2	80.5
5k	Line KX1, KY1, KX2, KY2 (CONSTs)	86.1	96.2	83.4	85.6	74.3

Test		RP2040	RP2040 off	RP2350	RP2350 off	Compiled
5l	Line ix1%, iy1%, ix2%, iy2% (integer variables)	85.9	95.4	82.9	85.0	74.1
6a	a = b + c written in place	10.2	13.6	11.4	12.1	2.12
6b	Sub0 (no parameters, empty)	13.5	14.8	10.4	11.6	4.96
6c	AddIt b, c	34.3	49.9	30.8	36.8	12.9
6d	Call "AddIt", b, c	41.9	55.2	38.6	43.4	30.0
6e	Call f$, b, c	40.7	55.2	37.6	42.5	29.0
6f	a = FAdd(b, c)	44.8	62.5	40.3	47.7	19.0
6g	a = Call("FAdd", b, c)	64.7	82.6	49.9	55.8	41.0
6h	GoSub AddG	13.9	19.0	17.1	19.1	6.42
6i	SubLoc (three LOCALs, otherwise empty)	32.5	39.7	27.2	32.3	14.2
7a	a = b + c	10.2	13.6	11.4	12.1	2.12
7b	16-17 letter names	10.3	18.8	11.5	16.2	2.12
7c	30 letter names	10.3	23.3	11.5	19.9	2.12
7d	AddTwoNumbersAndStoreTheResult b, c	34.7	55.1	30.9	40.0	12.9
7e	a = RGB(rr, gg, bb)	22.3	23.5	20.5	20.8	8.27
7f	RGB(with 26-28 letter names	54.3	55.5	40.9	41.2	8.27
8a	a = b + c (float)	10.2	13.6	11.4	12.1	2.12
8b	ia% = ib% + ic%	9.87	13.6	11.3	12.3	2.10
8c	a = b * c	10.4	13.8	11.4	12.2	2.18
8d	ia% = ib% * ic%	9.93	13.7	11.4	12.3	2.05
8e	a = b / c	10.6	14.0	11.6	12.4	2.69
8f	ia% = ib% \ ic%	9.97	14.7	11.6	12.6	2.63
8g	a = b ^ 2	10.8	14.5	11.1	11.6	3.16
8h	a = b * b	10.4	13.8	11.4	12.2	2.18
8i	a = Sqr(b)	10.6	12.9	11.9	12.4	2.41
8j	a = Sin(b)	14.4	16.6	12.8	13.2	3.00
9a	loop in a SUB on global variables	10.3	14.1	11.5	12.4	2.17
9b	loop in a SUB on LOCAL variables	10.2	14.0	11.3	12.8	2.17
9c	loop in a SUB on LOCAL INTEGERs	9.07	12.9	10.8	12.4	1.82
10a	a = x(5) + y(5)	16.0	21.0	17.3	18.8	2.89
10b	a = x(q) + y(q)	22.0	29.3	21.5	23.5	4.01
10c	a = m2(5, 5) + m2(6, 6)	20.6	26.2	22.0	23.8	3.70
11a	a = b	6.85	9.14	8.24	8.74	1.87
11b	a = b with a trailing ' comment	8.29	10.6	9.88	10.4	1.87
11c	a ' comment line and a = b	7.22	9.50	8.64	9.15	1.92

Test		RP2040	RP2040 off	RP2350	RP2350 off	Compiled
11d	a REM line and a = b	9.04	11.3	11.2	11.7	2.49
12a	single-line IF b < c THEN a = 1	15.7	18.0	16.5	17.1	2.15
12b	multi-line IF / a = 1 / ENDIF	12.9	15.2	13.6	14.1	2.15
12c	IF / 7 x ELSEIF, the last one true	74.7	87.3	71.1	73.8	7.32
12d	SELECT CASE, 8 cases, the last one true	31.6	36.9	32.8	33.4	7.68
13a	s$ = t$ + u$	11.8	15.2	13.4	14.2	5.20
13b	s$ = Str$(b)	37.3	55.7	24.0	24.5	19.3
13c	s$ = Left$(t$, 3)	15.9	18.2	16.5	17.0	9.02
14a	sum 1000 elements in a FOR loop	15.7	21.9	16.7	18.7	3.06
14b	the same with Math(SUM arr())	0.38	0.38	0.12	0.12	0.11
14c	set 1000 elements in a FOR loop	12.2	14.9	12.4	13.0	2.91
14d	the same with Math Set 1, arr()	0.54	0.54	0.15	0.15	0.15
14e	draw 1000 pixels in a FOR loop	28.3	37.5	28.5	30.9	9.11
14f	the same with Pixel px%(), py%()	0.28	0.29	0.32	0.32	0.30
15a	Inc a	5.59	6.74	5.96	6.21	2.10
15b	a = a + 1	9.95	12.2	10.4	10.9	2.12
15c	Inc ia%	5.58	6.84	6.01	6.33	2.03
15d	ia% = ia% + 1	9.14	11.7	10.2	10.8	2.03
16a	a=b+c (no spaces)	9.57	13.0	10.7	11.5	2.12
17a	a = Sin(b) * c + Sin(b) * d	31.6	37.3	26.0	27.2	4.92
17b	sx = Sin(b) : a = sx * c + sx * d	29.5	37.8	28.5	30.6	4.57

perfmath.bas: µs per element for the M tests, µs per sum (loop included) for the S tests. RP2350 is OPTION COMPILE OFF:

Test		RP2040	RP2350	Compiled
M1a	scale: b(j) = a(j) * 2.5 in a loop	21.7	21.1	4.16
M1b	Math Scale a(), 2.5, b()	0.60	0.22	0.22
M2a	offset: b(j) = a(j) + 7 in a loop	20.0	20.3	4.08
M2b	Math Add a(), 7, b()	0.84	0.24	0.23
M3a	c(j) = a(j) * b(j) in a loop	25.8	26.5	5.11
M3b	Math C_Mul a(), b(), c()	0.59	0.20	0.20
M4a	copy: b(j) = a(j) in a loop	17.3	18.4	3.85
M4b	b() = a()	0.03	0.03	0.03
M4c	Memory Copy Float	0.06	0.06	0.05
M5a	largest and where: IF in a loop	23.6	24.6	3.13
M5b	mx = Math(MAX a(), ix%)	0.30	0.15	0.14

Test		RP2040	RP2350	Compiled
M6a	mean in a loop	15.7	16.7	3.07
M6b	mn = Math(MEAN a())	0.36	0.12	0.12
M7a	dot product in a loop	24.3	24.8	4.34
M7b	tot = Math(DOTPRODUCT a(), b())	0.86	0.26	0.25
M8a	rotate 1000 points in a loop	86.5	79.1	12.1
M8b	Math V_Rotate 160, 120, 0.3, xi(), yi(), xo(), yo()	4.50	1.28	1.27
M9a	fit to 0-239 in a loop (find min/max, then scale)	67.4	69.7	9.47
M9b	Math Window a(), 0, 239, b()	2.98	0.95	0.94
M10a	clamp to 100-900 in a loop	52.8	54.9	6.68
M10b	Math Clamp a(), 100, 900, b()	0.53	0.23	0.22
M11a	Shell sort of 1000 numbers in BASIC	764	840	118
M11b	c() = a() : Sort c()	134	65.7	65.7
S4a	sum of 4 elements in a loop	81.7	90.8	16.4
S4b	sum of 4 elements with Math(SUM)	18.4	15.8	13.9
S16a	sum of 16 elements in a loop	271	295	52.9
S16b	sum of 16 elements with Math(SUM)	22.8	17.3	15.3
S64a	sum of 64 elements in a loop	1023	1099	200
S64b	sum of 64 elements with Math(SUM)	39.0	21.9	19.9
S4c	s4(0) + s4(1) + s4(2) + s4(3) written out	28.2	29.2	4.11
S0	the empty 2000-pass loop these use	2.17	2.29	1.12

verify.bas: the FOR loop ran 5 times on both boards. 4000 numbers took 2156 ms with `SORT` and 3632 ms with the BASIC Shell sort on the RP2040 (interpreted), and 1066 ms and 549 ms on the RP2350 (compiled).

Appendix B. Running the tests yourself

The programs are in the MMBasic source tree, in `Bas/bench/perfguide/`:

- `perfguide.bas`: the tests of sections 3 and 4, and the drawing tests of section 5;
- `perfguide_off.bas`: the same, with `OPTION SYMBOLS OFF`;
- `perfmath.bas`: the loop and MATH comparisons of section 5;
- `verify.bas`: the FOR-limit check of section 3.4, and the 4000-element sorts.

They are generated by `make_perfguide.py` and `make_perfmath.py`. Each test prints one line, `R <id> <passes> <best ms>`, so the time per pass in microseconds is best ms x 1000 / passes. The programs start with `OPTION CONSOLE SERIAL`, so the results go to the serial console. Delete that line to see them on the screen.

On an RP2350, run `perfguide.bas` once with `OPTION COMPILE OFF` and once with `OPTION COMPILE ON`. The raw results behind this guide are in `Bas/bench/perfguide/results/`.